

Deep RL 평가의 통계적 절벽

Deep Reinforcement Learning at the Edge of the Statistical Precipice

Rishabh Agarwal · Max Schwarzer · Pablo Samuel Castro · Aaron Courville · Marc G. Bellemare
(2021)

적은 run으로 만든 순위에서 불확실성을 읽고, 더 신뢰할 수 있게 보고하는 방법

적은 run이 통계적 절벽이 되는 이유

계산비용이 표본 수를 제한하면, 한 번의 점 추정치가 알고리즘 순위를 좌우한다.

01 비싼 학습 게임마다 긴 학습 반복



02 3-10 runs 관행상 작은 표본



03 점 추정 mean·median 한 숫자



04 불안정한 순위 다시 뽑으면 결론 변화

점수 차이만 보지 말고, 그 차이가 표본 변동에 비해 얼마나 큰지 함께 봐야 한다.

run은 seed 하나와 같지 않다

seed는 난수의 일부를 통제하지만, 독립 run의 변동 원인을 모두 제거하지 않는다.

변동 원인	예	해석
환경	전이·보상 stochasticity	같은 정책도 trajectory 변화
학습	탐색·초기화·minibatch	최적화 경로 변화
시스템	GPU·framework nondeterminism	같은 seed도 결과 변화 가능
실험 단위	독립적으로 다시 학습	seed 번호보다 생성 절차가 중요

seed를 고정했다는 말만으로 run의 독립성이나 결정성이 보장되지는 않는다.

Atari 100k 사례의 실험 단위

논문은 작은 표본의 영향을 보기 위해 알고리즘·게임마다 100개 run을 새로 모았다.

축	조건	의미
task	26 Atari games	게임마다 별도 score 분포
budget	100k environment steps	약 2-3시간 gameplay
run	알고리즘·게임당 100	작은 N을 재표집할 reference
evaluation	학습 뒤 100 episodes	각 run의 최종 score 산출
methods	DER·OTR·DrQ·CURL·SPR	DrQ 표준 epsilon도 구분

뒤의 수치는 이 26×100 구조에서 얻은 조건부 증거다.

점 추정치에도 분포가 있다

같은 알고리즘이라도 어떤 run을 선택했는지에 따라 aggregate score가 달라진다.

01 100-run pool 게임별 reference data



02 N개 재표집 N=3부터 100



03 통계량 계산 mean·median·IQM



04 100,000회 반복 sampling distribution

표 하나의 점수는 sampling distribution에서 뽑힌 한 관측값이다.

평가 시점이 다르면 숫자도 다르다

final policy 평가와 여러 시점·설정 중 maximum 선택은 서로 다른 질문이다.

protocol	선택 방식	답하는 질문
final	학습 종료 뒤 한 policy	마지막 policy의 성능
during-training max	여러 평가 시점 중 최고	관찰된 최고 checkpoint
configuration max	여러 hyperparameter 중 최고	탐색까지 포함한 최고값
비교 조건	같은 선택 규칙 적용	selection bias를 맞춰야 함

평가 protocol을 맞추지 않은 score 차이는 알고리즘 차이와 선택 효과를 섞는다.

통계량·구간·검정은 다른 층이다

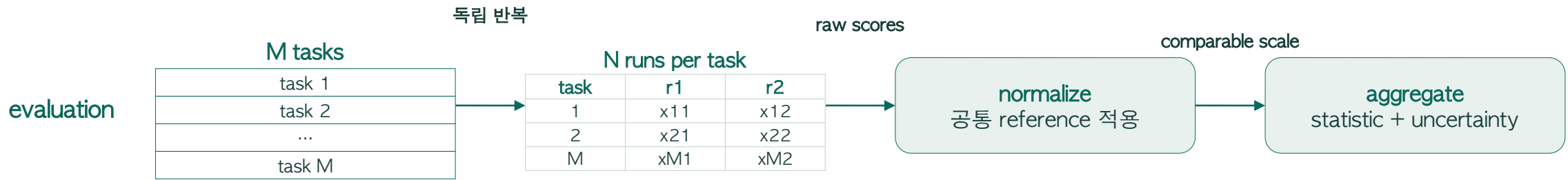
한 숫자, 그 숫자의 불확실성, 가설 판단 규칙을 분리해 읽는다.

층	출력	대답하는 질문
statistic	IQM=0.52	이번 표본의 중심은 얼마인가
confidence interval	95% CI [0.45,0.59]	반복 절차의 변동 범위는 어떤가
hypothesis test	검정통계량·결정	정한 귀무가설과 규칙에서 기각하는가
effect comparison	차이와 차이의 CI	두 방법의 차이는 얼마나 불확실한가

CI를 p-value로 읽거나, 점 추정치 하나를 가설의 결론으로 읽으면 안 된다.

평가 데이터는 $M \times N$ score matrix다

M개 task의 층을 유지한 채, 각 task 안에 N개의 독립 run을 둔다.



task와 run을 뒤섞지 말고, 두 수준의 구조를 통계 절차에 보존한다.

정규화와 metric 방향을 먼저 확인한다

같은 숫자라도 reference와 방향이 다르면 비교 의미가 달라진다.

metric	좋은 방향	해석
mean·median·IQM	높을수록	normalized score의 중심
optimality gap	낮을수록	목표보다 부족한 평균량
performance profile	곡선이 높을수록	threshold를 넘는 run 비율
human normalization	random=0, human=1	reference 선택이 scale을 결정
min-max normalization	task별 범위 사용	Procgen 비교에서 사용

점수 표를 읽기 전에 normalization reference와 good direction을 적는다.

mean과 median의 다른 취약점

robustness와 statistical efficiency는 같은 성질이 아니다.

statistic	장점	주의점
mean	모든 score 크기를 반영	극단값에 크게 이동
median	극단값에 robust	작은 N에서 분산·bias가 큼
IQM	중앙 50%의 크기 반영	tail 정보를 의도적으로 버림
선택 원칙	여러 관점을 함께 보고	분포와 질문에 맞춰 해석

한 metric을 고집하지 말고, 불확실성과 여러 관점을 함께 본다.

IQM은 중앙 50%의 평균이다

Interquartile Mean은 task-run score를 합쳐 양쪽 25%를 제거한 뒤 평균낸다.

01 normalized scores 모든 task-run 결합



02 sort 낮은 값부터 정렬



03 trim 하위·상위 25% 제거



04 average 중앙 50% 평균

IQM은 extreme tail의 영향은 제한하면서 중앙 score의 크기는 사용한다.

계산 예: outlier와 IQM

아래 숫자는 정의를 설명하기 위해 만든 예시이며 논문의 실험값이 아니다.

data	mean	IQM
0.0, 0.1, 0.2, 0.3, 0.8, 0.9, 1.0, 4.0	0.9125	0.55
중앙 네 값: 0.2, 0.3, 0.8, 0.9	—	2.2 / 4
4.0을 20.0으로 교체	2.9125	0.55
관찰	outlier에 이동	이 예시에서는 불변

robustness는 outlier의 영향을 제한한다는 뜻이며, 모든 정보 손실이 없다는 뜻은 아니다.

Performance profile은 분포를 곡선으로 본다

각 threshold를 넘는 task-run score의 비율을 계산해, 한 숫자가 숨긴 분포를 드러낸다.

01 threshold tau 관심 score 수준 설정



02 count score가 tau 초과인 run



03 divide 전체 $M \times N$ 으로 나눔



04 sweep tau tail curve 생성

곡선이 더 높으면 해당 threshold 이상을 달성한 run의 비율이 더 크다.

계산 예: profile이 교차할 수 있다

아래 score는 교육용 예시다. 비율은 threshold를 엄격히 초과한 run만 센다.

threshold	A: 0.2,0.7,1.2,1.5	B: 0.4,0.6,0.9,1.8	비교
0.5	$3/4 = 0.75$	$3/4 = 0.75$	동일
1.0	$2/4 = 0.50$	$1/4 = 0.25$	A가 높음
1.5	$0/4 = 0.00$	$1/4 = 0.25$	B가 높음
전체 곡선	중간 threshold 우세	높은 tail 우세	교차

profile이 교차하면 하나의 알고리즘이 모든 threshold에서 우세하다고 말할 수 없다.

Stratified bootstrap은 task 안에서 뽑는다

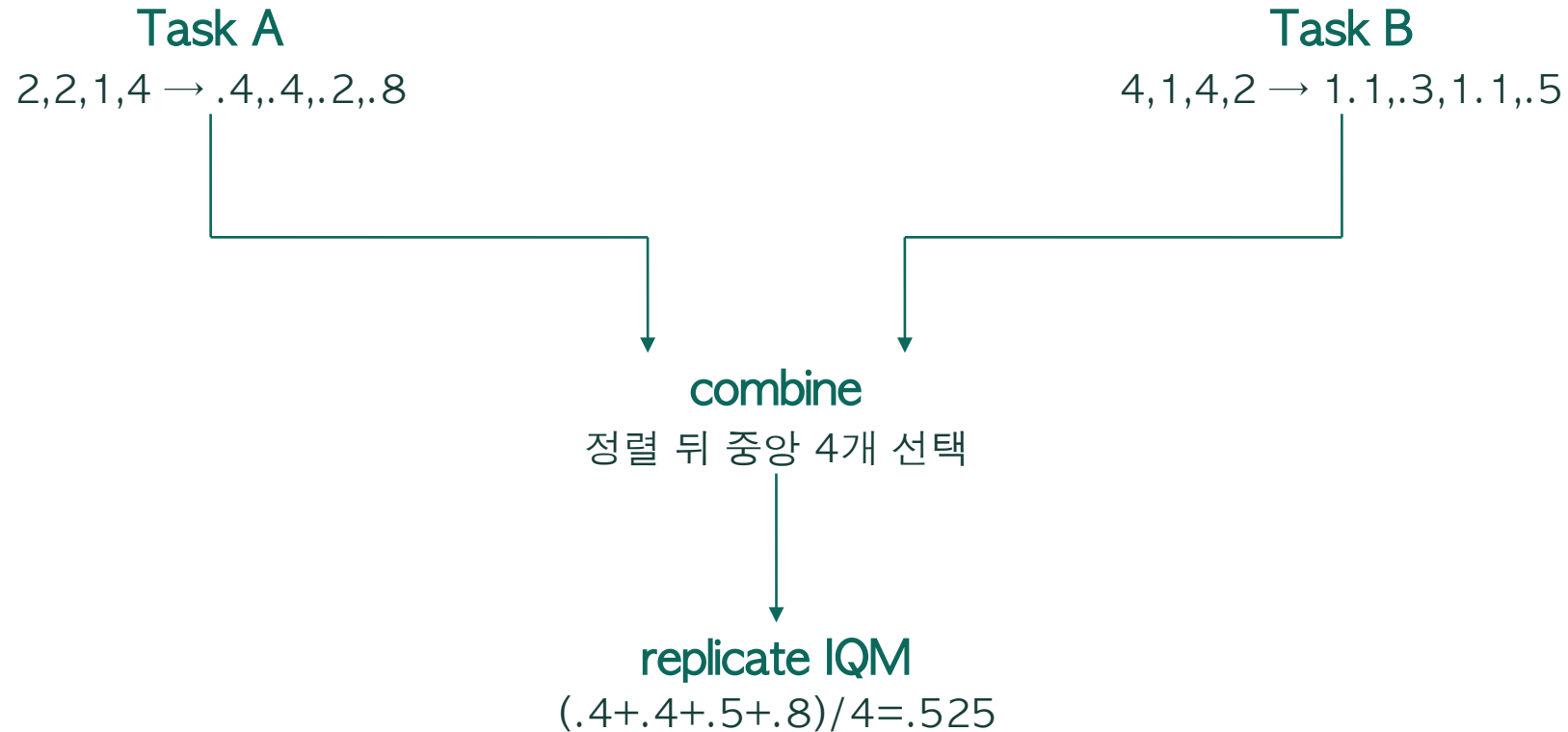
task 수와 run 수를 유지하면서 각 task의 run을 복원추출한다.

01	task별 scores	각 층에 N runs
↓		
02	within-task sample	복원추출로 N개
↓		
03	aggregate	IQM·median 재계산
↓		
04	repeat	bootstrap distribution
↓		
05	percentiles	confidence interval

모든 score를 한 통에서 뽑으면 task 구조가 사라지므로 논문의 절차가 아니다.

계산 예: bootstrap replicate 하나

교육용 두-task 예시다. 같은 task 안에서만 index를 복원추출한다.



95% CI를 올바르게 읽는 법

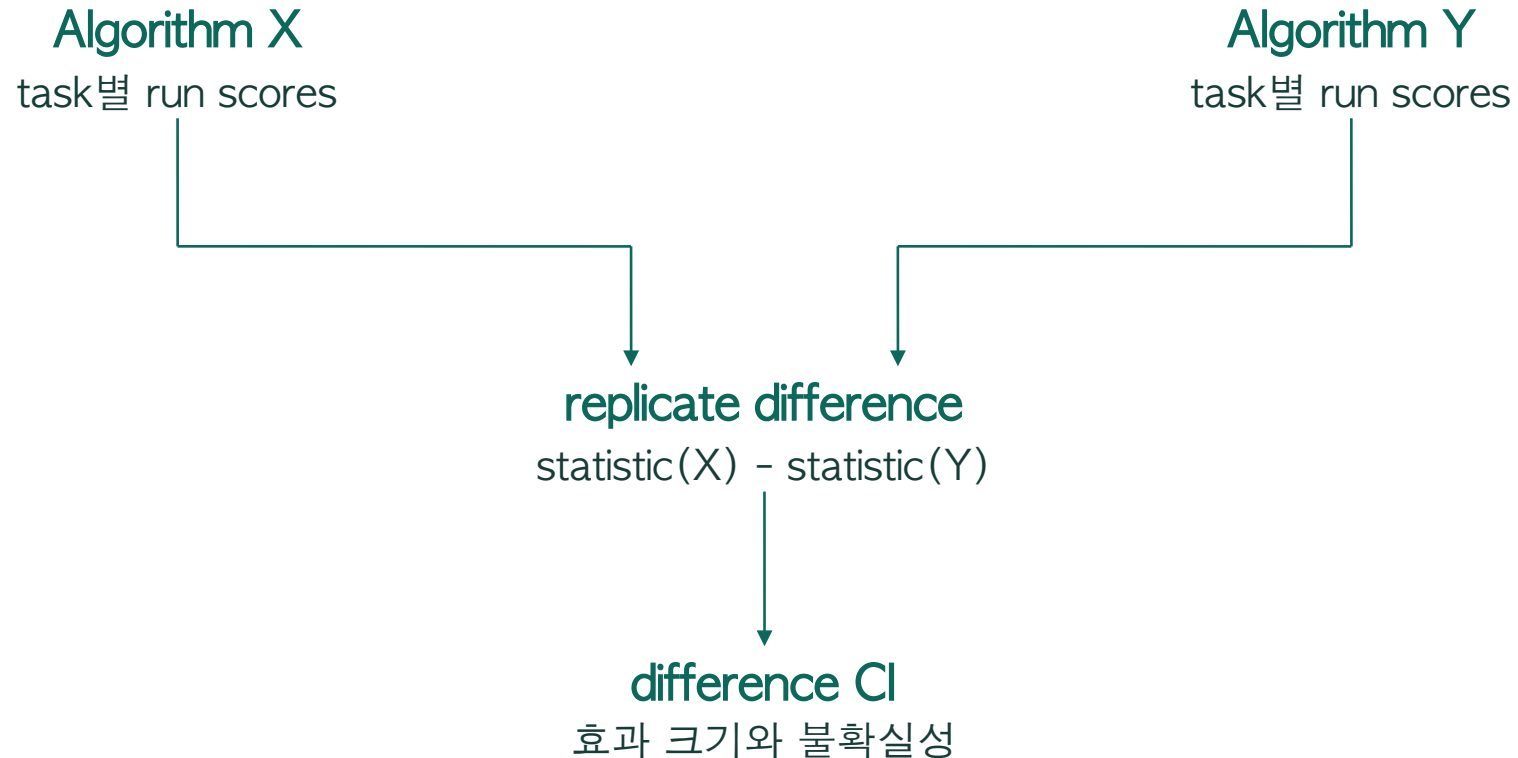
frequentist confidence는 고정된 참값의 사후확률이 아니라 절차의 장기 coverage다.

표현	판정	이유
이 절차를 반복하면 약 95%가 참값을 포함	적절	nominal coverage의 정의
참값이 이 구간 안일 확률은 95%	부적절	참값은 고정, interval이 random
구간이 좁으니 설계도 타당	부적절	bias·protocol 문제는 별도
N이 작아도 nominal coverage 보장	부적절	bootstrap도 small-N 한계

CI는 uncertainty를 보여 주지만, 잘못된 평가 protocol이나 bias를 자동으로 고치지 않는다.

알고리즘 비교에는 차이의 CI가 필요하다

각 알고리즘의 interval 두 개보다, bootstrap replicate마다 계산한 score 차이가 직접적인 대상이다.



Probability of improvement는 승률이다

task마다 X의 임의 run이 Y의 임의 run보다 클 확률을 구하고 평균낸다.

01 one task X run과 Y run pairing



02 compare $X > Y$ 는 1, tie는 0.5



03 task probability 모든 pair 평균



04 across tasks task 확률의 평균

0.7은 X가 얼마나 크게 이겼는지가 아니라, 임의 run 비교에서 이길 빈도를 뜻한다.

계산 예: 두 task의 개선 확률

아래 숫자는 교육용 예시다. 각 task에서 X run과 Y run의 모든 pair를 비교한다.

task	X runs	Y runs	X wins	확률
1	0.8, 1.0	0.6, 1.1	2 / 4	0.50
2	0.5, 0.7	0.4, 0.6	3 / 4	0.75
average	—	—	(0.50+0.75)/2	0.625
tie rule	X=Y	같은 score	0.5로 계산	크기와 무관

평균 개선 확률 0.625는 승리 빈도이며, 개선량 62.5%를 뜻하지 않는다.

Optimality gap은 목표 미달을 본다

target보다 낮은 score의 shortfall만 평균하며, 낮을수록 좋다.

score x	target 1과 비교	gap contribution
0.4	1보다 0.6 낮음	0.6
0.9	1보다 0.1 낮음	0.1
1.2	target 초과	0
해석	평균 shortfall	작을수록 target에 가까움

target 이상 개선을 0으로 자르므로, target 선택은 무엇을 중요하게 보는지 포함한다.

권장 reporting packet

하나의 leader-board 숫자보다 중심·불확실성·분포·직접 비교를 함께 제시한다.

01	protocol	task·runs·budget·selection
↓		
02	aggregate	IQM + percentile CI
↓		
03	distribution	performance profile + band
↓		
04	comparison	difference·improvement probability
↓		
05	raw context	per-task scores·normalization

여러 보기는 중복이 아니라 서로 다른 실패 모드를 드러내는 보완 관계다.

Atari 100k 근거의 실제 비용

대규모 reference dataset을 만들었기 때문에 작은-run 평가를 반복해서 분석할 수 있었다.

항목	조건	규모
main collection	6 variants $\times$ 26 games $\times$ 100 runs	15,600 runs
extra DER	100 runs $\times$ 26 games	2,600 runs
total	Tesla P100, run당 약 3-5시간	18,200 runs
estimated compute	학습시간 합산	약 2,400-3,600 GPU days

비싸게 모은 큰 표본으로 small-N 평가의 위험을 측정했다.

조건부 결과: median보다 IQM이 안정적이었다

Figure 2의 비교는 알고리즘별 원 논문 run 수를 반영한 Atari 100k subsample이다.

algorithm group	N in Fig. 2	관찰
DER·OTR·DrQ	5	median과 IQM 모두 넓은 변동
SPR	10	IQM distribution이 더 집중
CURL	20	metric에 따라 순위 해석 변화
50 runs까지	median	상당한 uncertainty가 남음
적용 범위	Atari 100k data	universal rule 아님

이 연구에서는 IQM이 더 효율적이었지만, 필요한 N은 분포와 질문에 따라 달라진다.

작은 개선일수록 더 많은 run이 필요했다

Figure 4는 DER score를 일정 비율 높인 통제 실험에서 추정 오차를 측정한다.

score lift	median의 조건부 관찰	IQM 비교
25%	약 25 runs에서 방어 가능	더 적은 run으로 안정
10%	약 100 runs가 필요	median보다 효율적
0%	100 runs에도 약 20% deviation 가능	오차가 0이 되지 않음
해석	effect·metric·N의 함수	분포 의존

표본 수만 말하지 말고, 기대하는 effect size와 metric을 함께 말해야 한다.

Bootstrap도 작은 N에서는 점검해야 한다

Figure 6은 200-run DER reference로 nominal 95% interval coverage를 검증한다.

target	Atari 100k 관찰	경계
aggregate median	약 5–10 runs에서 near nominal	N=3은 undercoverage
aggregate IQM	약 5–10 runs에서 near nominal	분포 의존
single-game mean	약 20–30 runs가 필요	aggregate와 다름
결론	procedure를 simulation으로 검증	5 runs 보편 충분 아님

aggregate CI의 조건부 성공을 모든 task-metric의 sample-size 규칙으로 일반화하면 안 된다.

Atari 200M: metric이 순위를 바꾼다

55개 sticky-action game에서도 run 수와 extreme score가 aggregate ranking에 영향을 준다.

조건	paper setting	읽을 점
tasks	55 Atari games	sticky actions 사용
runs	대부분 5	M-IQN 3, DreamerV2 11
mean	JAMESBOND score 영향	extreme tail에 민감
IQM·median	ranking 차이	metric 관점을 함께 봐야 함

더 큰 benchmark도 작은 run 수와 heavy tail 문제에서 자유롭지 않다.

DeepMind Control: uncertainty가 결론을 제한한다

6개 task의 100k·500k-step 결과는 point estimate보다 interval과 rank distribution을 함께 본다.

측	조건	해석
tasks	6 control tasks	task 수가 작음
budgets	100k and 500k steps	budget별 비교 분리
normalization	maximum 1000	높을수록 좋음
intervals	대부분 percentile bootstrap	일부는 studentized CI
result	많은 CI overlap	확정 순위를 피해야 함

표본·task 수가 제한되면 rank 하나보다 가능한 rank 분포가 더 정직하다.

Procgen: claimed gain과 승률은 다르다

16개 easy-mode task와 10 runs에서 average probability of improvement를 다시 본다.

조건	paper setting	관찰
tasks	16 easy-mode tasks	task 평균으로 비교
runs	10 per task	uncertainty가 남음
normalization	task별 min-max	PPO 기준보다 선호
claimed gains	대개 50-70% 승률	강한 지배가 아님
UCB-DrAC vs PLR	약 40-50%	일관된 개선으로 보기 어려움

평균 score가 높아도, 임의 run에서 이길 확률은 0.5 근처일 수 있다.

이 논문이 보장하지 않는 것

평가 통계는 결과 보고를 개선하지만, 알고리즘과 실험의 모든 타당성을 대신하지 않는다.

경계	보장하지 않음	필요한 추가 판단
sample size	5-10 runs의 보편적 충분성	분포·effect별 power 확인
reproducibility	method 구현의 완전 재현	code·환경·protocol 공개
confidence interval	bias 없는 설계와 인과 결론	selection·normalization 점검
one metric	모든 위험 신호의 순위	IQM·profile·gap 병행
fixed seed	결정성과 독립성	전체 randomness 통제

좋은 통계는 좋은 실험 설계의 대체재가 아니라, 그 결과를 정직하게 읽는 도구다.

Deep RL 결과표를 읽는 순서

숫자를 보기 전에 실험 단위와 비교 조건을 고정하고, 마지막에 불확실성을 확인한다.

- task 수 M, 독립 run 수 N, 학습 budget과 evaluation 시점을 확인한다.
- normalization reference와 metric의 좋은 방향을 먼저 적는다.
- IQM 같은 중심 통계와 confidence interval을 함께 읽는다.
- performance profile로 threshold별 분포와 curve 교차를 확인한다.
- 차이의 CI와 probability of improvement로 직접 비교한다.
- 결론을 해당 benchmark·run budget·protocol의 범위 안에 둔다.

핵심은 더 복잡한 숫자가 아니라, 결론이 건딜 수 있는 범위를 명시하는 것이다.